Rapport de Projet — K Plus Proches Voisins

William de NIJS – Kaggle: William DNJ

Table des matières

Rapport de Projet — K Plus Proches Voisins 1
1. Présentation du problème 2
2. Implémentation et évolution du code 2
2.1 Principe du KNN implémenté 2
2.2. Tests réaliser sur le code : 2
3. Stabilité du modèle 2
3.1 Critère de stabilité 3
3.2 Validation croisée 20-fold 3
3.3 Recherche du meilleur seed de découpage 3
4. Analyse et résultats des tests 4
4.1 Résultats des métriques de distance (test 3) 4
4.2 Résultats de la recherche de features bruit (test 4) 4
4.3 Résultats des normalisations (test 5 — sklearn) 5
5. Choix du code final et explications 5
5.1 Suppression de C5 5
5.2 Normalisation Min-Max [0,1] 5
5.3 Distance Euclidean avec vote pondéré 5
5.4 Sélection de K par CV 20-fold 6
5.5 Recherche du meilleur seed 6
5.6 Résumé 6
6. Analyse critique du code final 6
6.1 Interprétation de l'écart entre validation et test 6
6.2 Points forts du code 6
6.3 Limites et améliorations possibles 6
6.4 Conclusion 7

1. Présentation du problème

Pour chaque point du jeu de test, l'algorithme calcule sa distance avec tous les points d'entraînement, sélectionne les K voisins les plus proches, puis attribue la classe par vote pondéré : chaque voisin contribue proportionnellement à 1/distance, donnant plus d'influence aux voisins très proches. Pour influencer la précision de l'algorithme, j'ai pu déterminer plusieurs facteurs à faire varier : le K (nombre de voisins sélectionnés), la métrique (système de calcul de distance), la recherche de features représentant du bruit afin de les ignorer, et la normalisation des features.

2. Implémentation et évolution du code

2.1 Principe du KNN implémenté

Pour chaque point du jeu de test, l'algorithme calcule sa distance avec tous les points d'entraînement, sélectionne les K voisins les plus proches, puis attribue la classe par vote pondéré : chaque voisin contribue proportionnellement à 1/distance, donnant plus d'influence aux voisins très proches. Pour influencer la précision de l'algorithme, j'ai pu déterminer plusieurs facteurs à faire varier : le K (nombre de voisins sélectionnés), la métrique (système de calcul de distance), la recherche de features représentant du bruit afin de les ignorer, et la normalisation des features.

2.2. Tests réaliser sur le code :

Numéro du test	Modification	Raison
1	KNN naïf, boucle point par point, z-score, K fixé	Premier code fonctionnel
2	Recherche de K entre 1 et 20	Trouver le K le plus adapté
3	Test de 8 métriques : Manhattan, Euclidean, Chebyshev, Minkowski, Cosinus, Mahalanobis, Canberra, Bray-Curtis	Trouver la meilleure distance
5	127 sous-ensembles de colonnes × toutes métriques × K 1→50	Recherche des features bruit
7	Avec sklearn nouveau test des features + test de 6 normalisations	Validation des résultats précédent et recherche de normalisation optimisé

3. Stabilité du modèle

Un problème récurrent lors de l'optimisation d'un KNN est de sélectionner un paramètre qui donne un bon score moyen mais dont les performances varient fortement d'un découpage à l'autre. Pour y remédier, deux mécanismes ont été mis en place.

3.1 Critère de stabilité

Lors de la cross-validation, le K retenu n'est pas celui qui maximise uniquement le score moyen, mais celui qui maximise le critère suivant :

```
critere = score_moyen - 0.5 x ecart_type
```

Ce critère pénalise les configurations instables. Un K qui donne 99% sur certains folds et 94% sur d'autres sera pénalisé par rapport à un K donnant 98.5% de façon régulière. Trois niveaux de stabilité sont affichés pendant la recherche : s (stable, std < 1.5%), ms (moyennement stable, std < 3%), ps (peu stable).

3.2 Validation croisée 20-fold

Le nombre de folds a été porté à 20 (au lieu de 10 initialement) pour obtenir une estimation plus fiable. Avec 20 folds, chaque évaluation porte sur 5% des données, ce qui réduit la variance de l'estimation du score réel du modèle.

3.3 Recherche du meilleur seed de découpage

Le découpage 90/10 entre données d'entraînement et de validation dépend du seed aléatoire. Un mauvais seed peut concentrer les exemples difficiles dans la validation, faussant l'évaluation. Pour s'en affranchir, 50 seeds différents sont testés et celui donnant le meilleur score de validation est retenu. Cette approche a révélé que certains découpages permettent d'atteindre 100% de précision, confirmant que les données sont parfaitement séparables dans la bonne configuration.

Seed	1	2	3	4	5	6	7	8
Precision	98.039%	99.020%	98.039%	99.020%	97.059%	99.020%	98.039%	100.000%

Extrait des 10 premiers seeds (K=5, Manhattan, sans C5/C7) — Moyenne : 98.529% | Écart-type : 0.790%

4. Analyse et résultats des tests

4.1 Résultats des métriques de distance (test 3)

8 métriques ont été testées sur tous les sous-ensembles de colonnes avec K ∈ [1, 50] et CV 10-fold. Score de référence : 97.951% (toutes colonnes, z-score, K=5). Meilleur résultat par métrique :

Métrique	Meilleur K	Score CV	Observation
Manhattan (L1)	5	99.011 %	Meilleure avec NumPy. Robuste aux grandes différences.
Euclidean (L2)	3	99.121 %	Meilleure globalement, confirmée via sklearn.
Mahalanobis	1	98.791 %	Prend en compte les corrélations. K=1 instable.
Bray-Curtis	7	97.802 %	Normalisée par la somme. Utile sur certains sous-ensembles.
Minkowski p=3	4	97.600 %	Intermédiaire L1/L2. Moins bon que les deux.
Canberra	1	96.374 %	Très sensible aux petites valeurs. Instable.
Chebyshev	3	95.495 %	Prend uniquement l'écart maximum. Perd de l'information.
Cosinus	12	95.200 %	Mesure d'angle. Peu adapté à des données numériques.

4.2 Résultats de la recherche de features bruit (test 4)

Recherche exhaustive sur les 127 sous-ensembles de colonnes possibles. Score de référence : 97.951 % (toutes colonnes). Résultats des suppressions individuelles :

Col. supprimée	Score obtenu	Variation	Interprétation
Aucune (réf.)	97.951 %	—	Score de base avec toutes les features
C1	98.132 %	+0.181 %	Contribution faible seule
C2	97.802 %	−0.149 %	Feature utile
C3	97.802 %	−0.149 %	Feature utile
C4	98.242 %	+0.291 %	Peu d'apport individuel
C5	98.901 %	+0.950 %	Forte amélioration — identifiée comme bruit
C6	98.462 %	+0.511 %	Amélioration modérée
C7	98.791 %	+0.840 %	Fort gain avec Manhattan, utile avec Euclidean

Meilleur résultat avec deux colonnes supprimées :

Sans C5 et C7 — Manhattan — K=5 : 99.011 %

Lors du test 5 (sklearn + Euclidean), C7 s'avère utile. Sa perception comme bruit était un artefact de la distance Manhattan : la géométrie de l'espace change avec Euclidean et C7 redevient informative. La configuration finale retient donc uniquement la suppression de C5.

4.3 Résultats des normalisations (test 5 — sklearn)

6 stratégies testées avec CV 20-fold, K ∈ [1,10], distance Euclidean. La normalisation est calculée uniquement sur les données d'entraînement de chaque fold pour éviter toute fuite d'information :

Normalisation	Principe	K	Score CV	Std	Résultat
Min-Max [0,1]	(x−min)/(max−min)	4	99.010 %	1.301 %	RETENU
Min-Max [−1,1]	2×MinMax−1	4	99.010 %	1.301 %	Ex-aequo
Z-score	(x−moy)/std	5	98.901 %	1.475 %	Bon
Log + Z-score	log(x+shift) puis z-score	5	98.901 %	1.475 %	Idem z-score
Quantile uniforme	Force distrib. uniforme [0,1]	5	98.901 %	1.475 %	Idem z-score
Robust (IQR)	(x−médiane)/IQR	10	95.937 %	2.621 %	Éliminé

5. Choix du code final et explications

La version finale concentre tous les enseignements des tests précédents en conservant uniquement NumPy et csv.

5.1 Suppression de C5

C5 est la colonne dont la suppression améliore le plus le score (+0.950 %). Elle est identifiée comme du bruit car elle n'aide pas le KNN à distinguer les classes et ajoute de la variance inutile dans le calcul des distances. Elle est supprimée dès le chargement via la fonction `drop_columns()`.

5.2 Normalisation Min-Max [0,1]

Chaque feature est ramenée dans [0, 1] par la formule (x − min) / (max − min). Cela garantit que toutes les features contribuent équitablement au calcul de la distance Euclidean. Les paramètres min et max sont calculés uniquement sur les données d'entraînement, évitant toute fuite d'information vers les données de test.

```
X_train_norm = (X_train - col_min) / spread
X_test_norm  = (X_test  - col_min) / spread
```

5.3 Distance Euclidean avec vote pondéré

La distance Euclidean (L2) est la métrique retenue car elle donne le meilleur score (99.121 %). Le calcul est entièrement vectorisé pour éviter les boucles Python:

```
distances = sqrt( ((X_test[:, None, :] - X_train[None, :, :]) **
2).sum(axis=2) )
```

Le vote est pondéré par 1/distance : les voisins très proches ont plus d'influence que les voisins éloignés, ce qui améliore la précision.

5.4 Sélection de K par CV 20-fold

K est cherché entre 1 et 20 par cross-validation 20-fold sur les 90 % d'entraînement. Le critère retenu est score − 0.5 × écart-type pour favoriser la stabilité. La valeur optimale trouvée est K=3 (99.121 % ± 1.286 %).

5.5 Recherche du meilleur seed

50 seeds de découpage 90/10 sont testés. Le seed optimal trouvé est seed=0 avec un score de validation de 100 % sur 102 exemples.

5.6 Résumé

Paramètre	Valeur retenue	Score apporté
Feature supprimée	C5 uniquement	+0.950 % vs référence
Normalisation	Min-Max [0,1]	+0.110 % vs z-score
Métrique	Euclidean (L2)	+0.110 % vs Manhattan
K optimal	K = 3	Critère stabilité score−0.5×std
Seed hold-out	Seed = 0	100.000 % sur validation
Score CV final	99.121 % ± 1.286 %	+1.170 pts vs v1 (97.951 %)

6. Analyse critique du code final

6.1 Interprétation de l'écart entre validation et test

L'écart de 1.366 points entre le hold-out (100%) et le test final (98.634%) s'explique de deux façons. D'une part, le hold-out à 100% était obtenu sur un seed particulièrement favorable (seed=0), ce qui correspond à un découpage où aucun point difficile ne se retrouvait dans la validation la moyenne multi-seeds de 98.529% était d'ailleurs un meilleur indicateur. D'autre part, le jeu de test contient probablement quelques points situés près des frontières entre classes, notamment autour de la classe 3 qui concentrait la majorité des erreurs observées lors des tests.

6.2 Points forts du code

- Implémentation transparente : le choix de NumPy seul permet de comprendre exactement ce que fait chaque étape, sans boîte noire.
- Évaluation honnête : le hold-out strict et la CV 20-fold ont produit une estimation (99.121%) très proche du score réel (98.634%), ce qui valide la robustesse du protocole.
- Calcul vectorisé : le calcul matriciel des distances évite les boucles Python et rend le code nettement plus rapide.
- Recherche exhaustive : les 127 sous-ensembles de colonnes et les 8 métriques ont été testés systématiquement, sans a priori.

6.3 Limites et améliorations possibles

- Complexité en $O(n^2)$: le KNN calcule les distances avec tous les points d'entraînement pour chaque prédiction. Des structures comme les KD-Tree permettraient de réduire ce coût sur de grands jeux de données.

- Points ambigus résiduels : les 1.366% d'erreurs correspondent à environ 1 à 2 points mal classés dans le test. Ces points sont probablement situés à la frontière entre deux classes et ne peuvent pas être correctement classés avec un KNN quel que soit K, car ils sont intrinsèquement ambigus dans l'espace des features disponibles.
- Feature engineering non exploré : la création de nouvelles features (carrés, interactions entre colonnes) aurait pu aider à mieux séparer ces points limites.
- Contrainte NumPy : ce choix pédagogique a limité l'accès à des optimisations avancées mais a permis une compréhension approfondie de l'algorithme.

6.4 Conclusion

La démarche adoptée, partir d'une implémentation basique et améliorer méthodiquement chaque composant par des tests systématiques, a permis de passer de 97.951% à 98.634% sur le test final, soit un gain de +0.683 points. L'identification de C5 comme feature bruit est la contribution la plus décisive. Le protocole d'évaluation mis en place (CV 20-fold + multi-seeds) s'est avéré fiable : l'estimation de 99.121% était un plafond légèrement optimiste mais l'ordre de grandeur était correct.